

Topic 7: Distributed Storage and File Systems

Topic Outline

1. Distributed file systems (DFS)
2. Google File System, HDFS
3. Object stores and cloud storage
4. CAP theorem

Distributed Storage and File Systems

Distributed storage systems allow **data to be stored across multiple machines (nodes)** while appearing to users as a **single unified storage system**. They are essential for handling **massive datasets, high availability, and fault tolerance**.

Large-scale platforms like **Google, Amazon, and Microsoft** rely heavily on distributed storage technologies.

Types of Distributed Storage and File Systems

1. Distributed File Systems (DFS)

Definition

A **Distributed File System (DFS)** is a file system that allows **files to be stored across multiple servers but accessed as if they were stored locally**.

In a DFS:

- Files are **split into blocks**
- Blocks are **distributed across multiple nodes**
- Users interact with the system **as if it is a single file system**

Key Features of DFS

1. Transparency

Users do not know where files are physically stored.

Example:

A file might be split across 10 machines, but the user accesses it like a normal file.

2. Fault Tolerance

If a node fails, another replica can provide the data.

This ensures **data availability even during hardware failures**.

3. Scalability

DFS systems can easily scale by **adding more nodes** to the system.

4. High Throughput

Large data operations can run **in parallel across multiple machines**.

Basic DFS Architecture

A typical distributed file system contains:

1. Metadata Server (Master)

Stores:

- File names
- Block locations
- Access permissions

2. Storage Nodes (Workers)

These nodes store the **actual data blocks**.

Example Workflow

1. A user uploads a 300MB file.
2. The DFS splits the file into blocks (e.g., 128MB blocks).
3. Each block is stored on different servers.
4. The metadata server records where each block is stored.

Advantages of DFS

- Large storage capacity
- High reliability
- Faster processing of big data
- Fault tolerance

Disadvantages

- Complex design
- Network dependency
- Metadata server may become a bottleneck

2. Google File System (GFS)

The **Google File System** is a distributed file system developed by **Google** to handle **large-scale data processing workloads**.

It was designed to support applications like:

- Search indexing
- Data analytics
- Large web crawls

GFS Architecture

GFS has three main components:

1. Master Server

The master manages:

- File namespace
- Metadata
- Mapping of files to chunks

There is typically **one master node**.

2. Chunk Servers

Chunk servers store the **actual data chunks**.

- Files are divided into **64MB chunks**
- Each chunk is stored on multiple servers

3. Client

Clients interact with the system by:

1. Requesting metadata from the master
2. Communicating directly with chunk servers

GFS Replication

Each chunk is **replicated 3 times** across different machines.

Benefits:

- Fault tolerance
- High availability

Example

Suppose a 200MB file is uploaded.

Process:

- File split into **4 chunks (64MB each approx)**

- Each chunk replicated **3 times**
- Stored on multiple servers

Advantages of GFS

- Optimized for large files
- High fault tolerance
- Supports massive data processing

3. Hadoop Distributed File System (HDFS)

HDFS allows data to be stored across a **cluster of computers** instead of a single machine. It splits large files into **blocks** and distributes them across different nodes in the cluster.

HDFS is widely used in **big data processing and analytics**.

HDFS Architecture

HDFS has three major components:

1. NameNode

The **NameNode** manages:

- File system metadata
- File structure
- Block locations

This is the **master node**.

2. DataNodes

DataNodes store the **actual data blocks**.

Functions:

- Store file blocks
- Handle read/write operations
- Report to NameNode

3. Client

The client:

- Requests metadata from NameNode
- Reads/writes data directly from DataNodes

HDFS Block Structure

Default block size:

- **128MB or 256MB**

Each block is **replicated across nodes**.

Default replication factor:

- **3 copies**

Example

Suppose a file is stored in **Hadoop Distributed File System (HDFS)** with:

- **File size = 512 MB**
- **Block size = 128 MB**
- **Default replication factor = 3**

Calculate Number of Blocks

- Number of Blocks = Block Size / File Size
= 512 / 128
= 4

So the file is divided into **4 blocks**:

- **B1 = 128 MB**
- **B2 = 128 MB**

- **B3 = 128 MB**
- **B4 = 128 MB**

Replication (Factor = 3)

Block	Copy 1	Copy 2	Copy 3
B1	DataNode 1	DataNode 2	DataNode 3
B2	DataNode 2	DataNode 3	DataNode 4
B3	DataNode 1	DataNode 3	DataNode 4
B4	DataNode 2	DataNode 4	DataNode 5

3. Total Block Copies Stored

So the cluster stores **12 block replicas**.

Total Copies = Blocks × Replication Factor = 4 × 3 = 12

*Each block is **replicated to other nodes** for reliability.*

Advantages of HDFS

- Handles **petabytes of data**
- Fault tolerant
- Optimized for **batch processing**

Limitations

- Not ideal for **small files**
- Single NameNode may become bottleneck
- Not designed for low-latency access

4. Object Stores and Cloud Storage

Object storage is another model used for **storing massive amounts of unstructured data**.

Instead of files and blocks, data is stored as **objects**.

Structure of Object Storage

Each object contains:

1. **Data**
2. **Metadata**
3. **Unique identifier**

Objects are stored in **flat address spaces** rather than hierarchical folders.

Example Object Storage Systems

Popular cloud object stores include:

- **Amazon S3**
- **Google Cloud Storage**
- **Microsoft Azure Blob Storage**

Example

A photo stored in an object store may look like:

Object ID: 873829373

Data: image.jpg

Metadata:

- size: 3MB

- upload date: 2026-03-10

- owner: user123

The object is retrieved using the **object ID**.

Advantages of Object Storage

- Highly scalable
- Suitable for large unstructured data
- Easy integration with cloud services

Common Use Cases

- Cloud backups
- Media storage
- Big data lakes
- Machine learning datasets

Example:

Companies store huge datasets in **Amazon S3** for analytics.

5. CAP Theorem

The **CAP Theorem** explains the fundamental trade-offs in distributed systems.

It states that a distributed system can guarantee **only two of the following three properties at the same time**.

1. Consistency (C)

All nodes see **the same data at the same time**.

Example:

If data is updated, every node immediately returns the updated value.

2. Availability (A)

Every request receives a **response**, even if some nodes fail.

Example:

Users can still access the system even if a server crashes.

3. Partition Tolerance (P)

The system continues to function **despite network failures between nodes**.

Example:

Two clusters of servers may temporarily lose communication but still operate.

CAP Trade-Off

A distributed system must choose between:

System Type	Meaning
CP	Consistency + Partition tolerance
AP	Availability + Partition tolerance
CA	Consistency + Availability (rare in distributed systems)

Real-World Examples

CP Systems

Prioritize **consistency**.

Example technologies:

- **HBase**
- **MongoDB** (in certain configurations)

AP Systems

Prioritize **availability**.

Example:

- **Apache Cassandra**
- **Amazon DynamoDB**

These systems may return **eventually consistent data**.

6. Comparison: DFS vs Object Storage

Feature	Distributed File System	Object Storage
Structure	Hierarchical files	Flat object storage
Access	File system interface	API-based access
Best For	Big data processing	Cloud storage
Example	Hadoop Distributed File System	Amazon S3

7. Real-World Example

A data analytics company processes **petabytes of user activity logs**.

Workflow:

1. Logs stored in **Hadoop Distributed File System**
2. Data processed using Hadoop ecosystem tools
3. Processed data stored in **Amazon S3** for long-term storage
4. System designed considering the **CAP Theorem** to balance availability and consistency.

Summary

Distributed File Systems

- Store data across multiple machines
- Provide reliability and scalability

Google File System

- Large-scale distributed file system developed by **Google**
- Uses master and chunk servers

HDFS

- Open-source DFS used in big data
- Uses NameNode and DataNodes

Object Storage

- Stores data as objects with metadata
- Used in cloud storage systems

CAP Theorem

A distributed system can only guarantee **two of three properties**:

1. Consistency
2. Availability
3. Partition tolerance

Topic review Question

A global weather monitoring system collects data from thousands of sensors worldwide. Each sensor sends data every minute. Daily data generated is 512 MB, block size of 128 MB and default replication factor of 3.

- i) Recommend and explain suitable distributed file systems for storing sensor data
- ii) Discuss how block distribution improve performance using an example with clear distributed block
- iii) Discuss means in which parallel processing can be used with this storage system

Topic 8: Distributed Resource Management

Topic Outline

1. Scheduling and load balancing
2. Distributed transactions and 2-phase commit
3. Distributed deadlocks
4. Resource allocation and fairness

Distributed Resource Management deals with **efficiently managing and allocating resources** (CPU, memory, storage, network bandwidth, etc.) across **multiple computers in a distributed system**. Its goal is to ensure **efficient utilization, fairness, reliability, and high performance**.

1. Scheduling and Load Balancing

Scheduling

Scheduling determines **which task should run on which machine and at what time** in a distributed system.

Objectives of Scheduling

- Maximize resource utilization
- Minimize execution time
- Improve system throughput
- Ensure fairness among tasks

Types of Scheduling

1. Static Scheduling

- Decisions made before execution.
- Based on known workload and resources.

Example: Assigning specific tasks to certain servers before starting a distributed computation.

2. Dynamic Scheduling

- Decisions made during execution.
- Adapts to system changes such as node failures or workload variations.

Example: Cloud systems assigning tasks to servers with the lowest current load.

Load Balancing

Load balancing distributes **work evenly across nodes** in a distributed system to avoid overload on one machine.

Goals

- Prevent resource bottlenecks
- Improve response time
- Increase system efficiency

Types

1. Static Load Balancing

- Task allocation predetermined.

2. Dynamic Load Balancing

- Tasks distributed according to current load conditions.

Example

In a web server cluster:

- Incoming user requests are distributed across several servers.
- Prevents one server from being overloaded.

2. Distributed Transactions and Two-Phase Commit (2PC)

Distributed Transactions

A distributed transaction is a **transaction that accesses and updates data across multiple networked databases or nodes.**

To maintain consistency, distributed transactions follow **ACID properties:**

- **Atomicity:** All operations complete or none do.
- **Consistency:** Database remains valid after transaction.
- **Isolation:** Transactions do not interfere with each other.
- **Durability:** Completed transactions persist even after failure.

Two-Phase Commit (2PC)

2PC is a **protocol used to ensure all nodes agree to commit or abort a transaction.**

Phase 1 – Prepare Phase

1. Coordinator sends **prepare request** to all participating nodes.
2. Each node checks if it can commit.
3. Nodes respond with:
 - **YES (ready)** or
 - **NO (abort)**

Phase 2 – Commit Phase

- If all nodes respond **YES**, the coordinator sends **COMMIT**.
- If any node responds **NO**, the coordinator sends **ABORT**.

Example

A bank transfer between two databases:

- Debit account in Database A.
- Credit account in Database B.
- Both must succeed or both must fail.

3. Distributed Deadlocks

Deadlock Definition

A **deadlock occurs when two or more processes wait indefinitely for resources held by each other.**

Example

- Process A holds **Resource 1** and waits for **Resource 2**.
- Process B holds **Resource 2** and waits for **Resource 1**.

Both processes become stuck.

Deadlock Handling Methods

1. Deadlock Prevention

Avoid situations that lead to deadlocks by restricting resource requests.

Example:

- Force processes to request all resources at once.

2. Deadlock Detection

System periodically checks for cycles in the **resource allocation graph**.

3. Deadlock Recovery

Once detected:

- Terminate one process, or
- Force resource release.

4. Resource Allocation and Fairness

Resource allocation ensures **efficient and fair distribution of system resources among processes or users.**

Goals

- Efficient utilization of resources
- Avoid starvation
- Maintain fairness among processes

Fairness Concepts

1. Equal Share

Each user or process gets an equal portion of system resources.

2. Priority-Based Allocation

Processes with higher priority receive resources first.

3. Weighted Allocation

Resources distributed based on assigned weights.

Example:

- Premium users receive more CPU time in a cloud service.

Challenges in Distributed Resource Allocation

- Resource heterogeneity (different types of hardware)
- Communication delays
- Node failures
- Scalability issues

Conclusion

Concept	Description
Scheduling	Decides where and when tasks run
Load Balancing	Distributes tasks evenly across nodes
Distributed Transactions	Transactions across multiple systems
Two-Phase Commit	Ensures consistent commit/abort decisions
Distributed Deadlocks	Processes waiting indefinitely for resources
Resource Allocation	Fair distribution of system resources

Topic review question

Two distributed processes are running on different machines. Process A holds Resource X and requests Resource Y. Process B holds Resource Y and requests Resource X.

Neither process releases its resource.

- Explain the problem illustrated in this scenario by the two processes.
- Explain the deadlock condition present based on this context
- Discuss techniques in which the system detect such deadlocks

Topic 9: Security in Distributed Systems

Topic Outline

1. Authentication and authorization
2. Encryption and secure communication
3. Trust and intrusion detection
4. Security in cloud and peer-to-peer systems

1. Authentication and Authorization

Authentication

Authentication is the process of **verifying the identity of a user, device, or system** before allowing access to resources.

Common Authentication Methods

1. **Password-based authentication**
 - Users log in using a username and password.
2. **Two-Factor Authentication (2FA)**
 - Combines two verification methods such as password + OTP.
3. **Biometric authentication**
 - Uses fingerprints, facial recognition, or iris scans.
4. **Digital certificates**
 - Used to verify identities in secure networks.

Example

When logging into an online banking system, the system verifies your **username, password, and sometimes an OTP** to confirm your identity.

Authorization

Authorization determines **what resources an authenticated user is allowed to access**.

Types of Authorization

1. **Role-Based Access Control (RBAC)**
 - Permissions are assigned based on user roles.
2. **Access Control Lists (ACLs)**
 - Lists which users can access certain resources.
3. **Attribute-Based Access Control (ABAC)**
 - Access is granted based on attributes such as user identity, time, and location.

Example

- A **system administrator** can modify server configurations.
- A **regular user** can only view files.

2. Encryption and Secure Communication

Encryption is the process of **converting data into an unreadable format** to prevent unauthorized access during transmission.

Types of Encryption

1. Symmetric Encryption

- Uses **one key** for both encryption and decryption.
- Faster but requires secure key sharing.

Example: AES encryption.

2. Asymmetric Encryption

- Uses **two keys**:
 - Public key (for encryption)
 - Private key (for decryption)

Example: RSA encryption.

Secure Communication Protocols

Secure protocols protect data transferred between nodes.

Examples include:

- **HTTPS** – Secure web communication
- **SSL/TLS** – Encryption for internet connections
- **SSH** – Secure remote login

Example

When visiting a website with **HTTPS**, the data between your browser and the server is encrypted.

3. Trust and Intrusion Detection

Trust in Distributed Systems

Trust determines **which nodes or users can be relied upon in the network.**

Trust Mechanisms

- Digital certificates
- Public Key Infrastructure (PKI)
- Reputation systems in peer networks

Example

A secure server verifies another server's identity using **digital certificates** before exchanging data.

Intrusion Detection Systems (IDS)

An **Intrusion Detection System** monitors network activities to **detect suspicious or malicious behavior.**

Types of IDS

1. **Network-based IDS (NIDS)**
 - Monitors network traffic for unusual patterns.
2. **Host-based IDS (HIDS)**
 - Monitors activity on individual computers.

Examples of Detected Threats

- Unauthorized access attempts
- Malware activity
- Denial-of-Service attacks

4. Security in Cloud and Peer-to-Peer Systems

Security in Cloud Systems

Cloud computing stores and processes data on **remote servers**, so security is very important.

Common Cloud Security Measures

- Data encryption
- Identity and access management
- Regular security audits
- Backup and disaster recovery

Cloud Security Risks

- Data breaches
- Unauthorized access
- Service outages

Security in Peer-to-Peer (P2P) Systems

In **P2P networks**, each node acts as both a client and server, which introduces unique security challenges.

Security Challenges

- Lack of centralized control
- Risk of malicious peers
- Data integrity issues

Security Solutions

- Encryption
- Reputation systems
- Digital signatures
- Peer authentication

Example

File-sharing networks verify file integrity using **hash values** to ensure files are not tampered with.

Conclusion

Topic	Description
Authentication	Verifies identity of users or systems
Authorization	Determines access rights
Encryption	Protects data from unauthorized access
Secure Communication	Uses protocols like HTTPS and TLS
Trust	Ensures nodes are reliable
Intrusion Detection	Detects malicious activities
Cloud Security	Protects remote data and services
P2P Security	Handles risks in decentralized networks

Topic10: Case Studies and Advanced Topics

Topic Outline

1. Peer-to-peer networks: BitTorrent, blockchain basics
2. Cloud computing: AWS, Azure, microservices
3. Big data frameworks: Hadoop, Spark
4. Emerging topics: edge computing, IoT systems

These are practical application of distributed systems usages in different fields across various systems and technologies. They include the following

1. Peer-to-Peer (P2P) Networks

A **peer-to-peer network** is a distributed architecture where **each computer (peer) can act as both a client and a server**.

Key Characteristics

- No central server
- Peers share resources directly
- Scalable and fault-tolerant

Example: BitTorrent

- Used for **large file distribution**
- Files are split into small pieces
- Users download and upload pieces simultaneously

Advantages

- High scalability
- Efficient bandwidth use
- No single point of failure

Challenges

- Security risks
- Difficult content control
- Dependence on active peers

2. Blockchain Basics

A **blockchain** is a **distributed ledger** that records transactions across many computers.

Key Concepts

- **Block:** Contains transaction data
- **Chain:** Blocks linked using cryptographic hashes
- **Decentralization:** No central authority
- **Consensus mechanisms:** Nodes agree on valid transactions

Popular Example

- Bitcoin

Why Blockchain Matters

- Trust without intermediaries
- Transparency
- Tamper-resistant records

Applications

- Cryptocurrencies
- Smart contracts
- Supply chain tracking
- Digital identity

3. Cloud Computing

Cloud computing provides **on-demand computing resources over the internet**.

Major Providers

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud

Cloud Service Models

1. **IaaS** – Infrastructure as a Service
Example: Virtual machines
2. **PaaS** – Platform as a Service
Example: application development platforms
3. **SaaS** – Software as a Service
Example: web apps

4. Microservices Architecture

Microservices break a large application into **small independent services**.

Features

- Each service performs **one specific function**
- Services communicate through APIs
- Independent deployment

Example Structure

Instead of one big app:

- Authentication service
- Payment service
- User service
- Notification service

Benefits

- Scalability
- Faster development
- Fault isolation

Challenges

- Service communication complexity
- Monitoring many services

5. Big Data Frameworks

Big data frameworks process **massive datasets distributed across many machines**.

Apache Hadoop

Components:

- **HDFS** – distributed file system
- **MapReduce** – parallel processing model

Used for:

- batch processing
- large-scale storage

Apache Spark

Faster than Hadoop MapReduce.

Key Features:

- In-memory processing
- Real-time analytics

- Machine learning libraries

Used for:

- streaming data
- AI pipelines
- large analytics workloads

6. Edge Computing

Edge computing processes data **closer to where it is generated**, instead of sending everything to the cloud.

Why It's Needed

- Reduce latency
- Lower bandwidth usage
- Improve real-time performance

Example

Self-driving cars analyzing sensor data **locally instead of sending everything to a remote server**.

7. Internet of Things (IoT) Systems

The **Internet of Things (IoT)** refers to **physical devices connected to the internet that collect and exchange data**.

Examples

- Smart thermostats
- Wearable fitness trackers
- Industrial sensors
- Smart cities

Components

1. Sensors/devices
2. Connectivity (Wi-Fi, cellular, etc.)
3. Data processing (cloud or edge)
4. Applications

How These Topics Connect

Modern systems often combine all of them:

Example architecture:

- **IoT devices** collect data
- **Edge computing** processes real-time data
- Data sent to **cloud platforms** like AWS/Azure
- **Spark/Hadoop** analyze large datasets
- **Microservices** deliver applications
- **Blockchain/P2P** provide decentralized trust

Conclusion

Topic	Purpose
P2P networks	Direct sharing between computers
Blockchain	Decentralized trusted records
Cloud computing	On-demand internet infrastructure
Microservices	Modular application design
Hadoop/Spark	Big data processing
Edge computing	Local data processing
IoT	Connected smart devices

REVIEW QUESTION

A city uses IoT sensors, edge computing, and cloud analytics to manage traffic. During a major festival, traffic data spikes massively and some sensors fail.

- i) If edge nodes fail in multiple locations, explain three strategies which maintains system functionality
- ii) Discuss three limitations of IoT-based traffic management in extreme conditions.
- iii) Discuss how blockchain technology enhance trust in multi-agency smart city projects